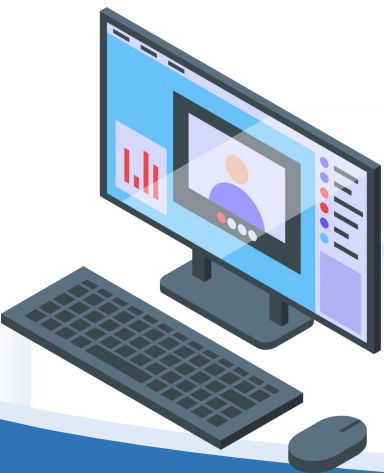
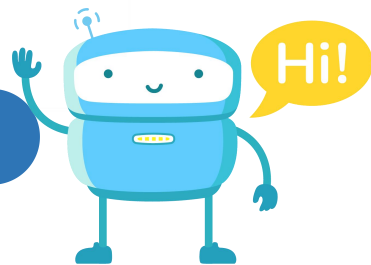


自然语言处理

第四章：中文分词与关键词提取 (一)



授课老师： 缪尧



参考资料：《自然语言处理实践（第二版）》、李轩涯主编、清华大学出版社、2023-12、ISBN：9787302649212。

《自然语言处理》、冯建周主编、中国水利水电出版社、2022-04、ISBN：9787522605272。

《Machine learning/Deep learning Course》Hung-yi Lee

目录

01

中文分词

02

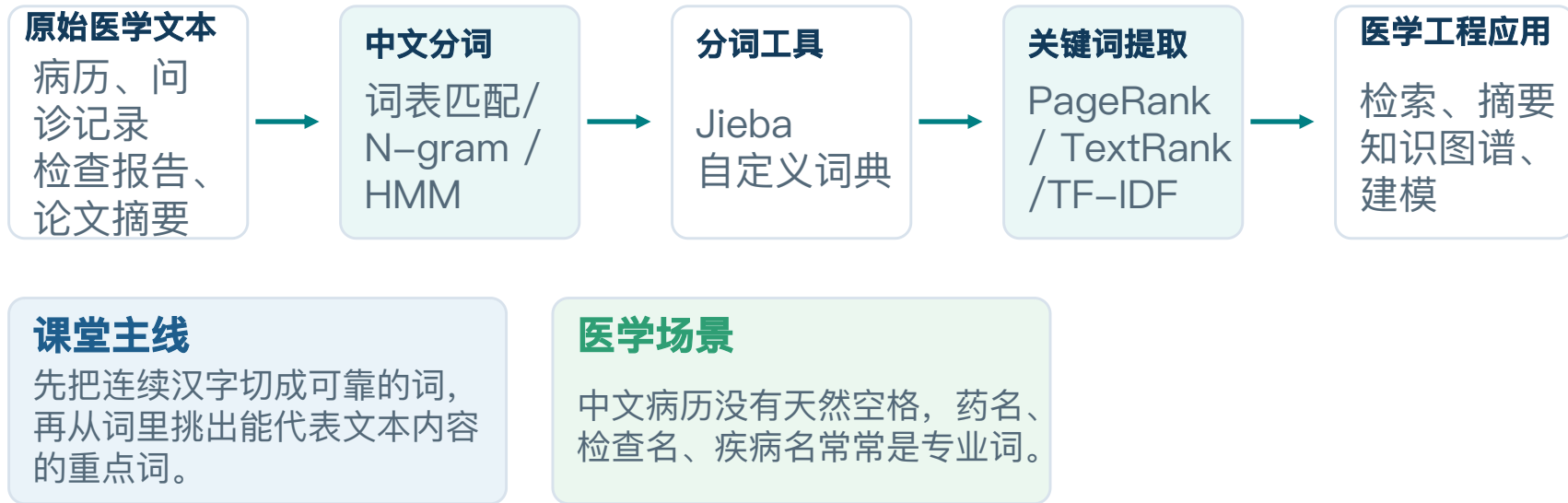
关键词提取

Part 01

中文分词

- 1.1 基于词表的分词算法
- 1.2 基于统计模型的分词算法
- 1.3 基于序列标注的分词算法
- 1.4 中文分词工具

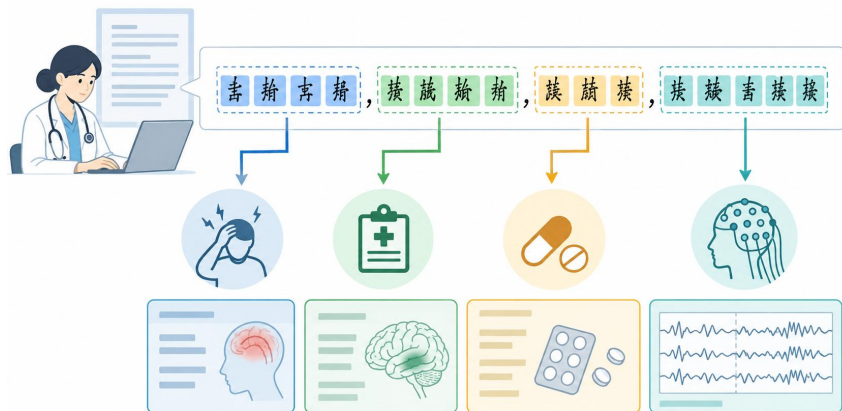
本章知识地图：两个任务，六类方法



先切词

FMM、RMM、BM 像查词典；N-gram 像看搭配概率；HMM 像给每个字贴 BMES 标签；Jieba 是常用工程工具。

案例导入：病历文本为什么必须先分词



原始句子

患者头痛伴恶心，脑电图提示左颞区异常放电。

患者 头痛 伴 恶心 脑电图 提示 左颞区 异常放电

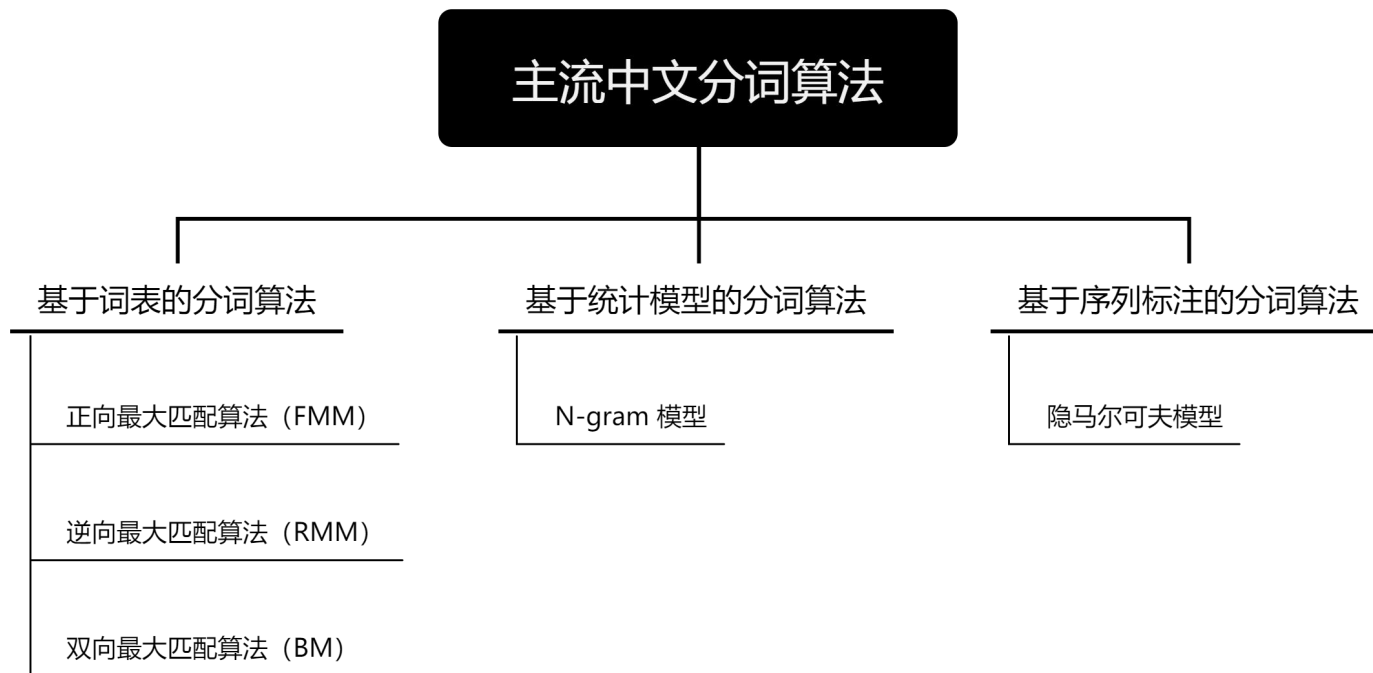
切错的风险

“左颞区异常放电”
如果被拆乱，后续
病灶定位和检索都
会受影响。

切对的价值

症状、检查、脑区
和疾病线索能进入
表格、图谱和模型。

主流的中文分词算法





01

基于词表的分词算法

1.正向最大匹配算法（FMM）- Forward Maximum Matching method

◀ **理解：** 从左向右扫描文本， 寻找词的最大匹配。

◀ **原理：**

正向最大匹配算法

(1) 比较词典中所有的词语，得到词典中最长词的长度为最大长度 max。

(2) 从待分词文本的最左端（句首）开始，从左向右进行扫描，每次扫描的字符长度为 max。

(3) 把扫描得到的字符串与词典中的词进行匹配

如果匹配成功，则切分出该词；

如果匹配失败，则去掉字符串最右边的一个字，
然后利用剩下的字符串重新和词典中的词进行匹配，
直到剩余字符串与词典中的词完全匹配。

(4) 当前字符串匹配完成后，接着按照最大长度 max 从左向右进行文本扫描、匹配，直到扫描到句尾，全部匹配成功。

1.正向最大匹配算法（FMM）——算法举例

【例】用正向最大匹配法对“秦皇岛今天晴空万里”进行中文分词，见下表。

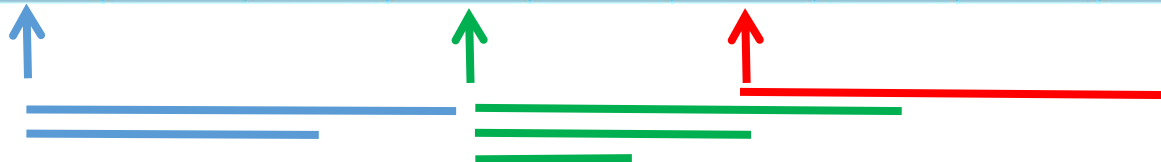
词典：“秦皇岛”“岛”“今天”“天晴”“晴空万里”“万里”……

text	0	1	2	3	4	5	6	7	8
文本	秦	皇	岛	今	天	晴	空	万	里

【案例分析】

根据当前词典，单词扫描的最大长度 $\max=4$ （实际的词典最大长度可能会更大）

0	1	2	3	4	5	6	7	8
秦	皇	岛	今	天	晴	空	万	里



秦皇岛

今天

晴空万里

1.正向最大匹配算法（FMM）——注意事项

- （1）这里要做到最大匹配，所以不一定第一次匹配成功就可以切分。
- （2）为提升扫描效率，也可以根据汉字数量的情况来设计多个词典，然后根据字数分别从不同词典中进行扫描。

2. 逆向最大匹配算法 (RMM) - Reverse directional Maximum Matching method

◀ 理解：从右向左扫描文本，寻找词的最大匹配。

◀ 原理：

逆向最大匹配算法

(1) 比较词典中所有的词语，得到词典中最长词的长度为最大长度 max。

(2) 从待分词文本的最右端（句尾）开始，从右往左进行扫描，每次扫描的字符长度为 max。

(3) 把扫描得到的字符串与词典中的词进行匹配

如果匹配成功，则切分出该词；

如果匹配失败，则去掉字符串最左边的一个字，
然后利用剩下的字符串重新和词典中的词进行匹配，
直到剩余字符串与词典中的词完全匹配。

(4) 当前字符串匹配完成后，接着按照最大长度 max 从右向左进行文本扫描、匹配，直到扫描到句首，全部匹配成功。

2. 逆向最大匹配算法 (RMM) —— 算法举例

【例】用逆向最大匹配法对“秦皇岛今天晴空万里”进行中文分词，见下表。

词典：“秦皇岛”“岛”“今天”“天晴”“晴空万里”“万里”……

text	0	1	2	3	4	5	6	7	8
文本	秦	皇	岛	今	天	晴	空	万	里

【案例分析】

根据当前词典，单词扫描的最大长度 $\max=4$ （实际的词典最大长度可能会更大）

0	1	2	3	4	5	6	7	8
秦	皇	岛	今	天	晴	空	万	里

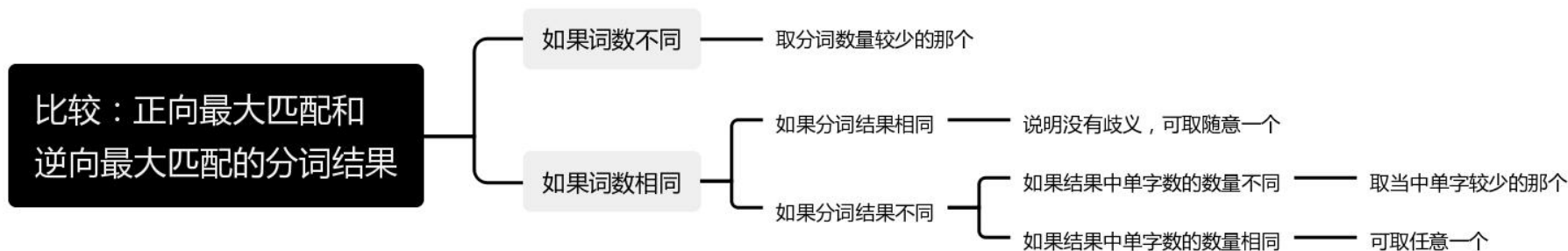


2. 逆向最大匹配算法（RMM）——注意事项

- （1）逆向最大匹配算法使用的分词词典是逆序词典，里面的每个词都将按逆序方式存放。在实际应用过程中，可以将待分词文本进行倒排处理，从而生成逆序文本，然后再根据逆序词典，对逆序文本用正向最大匹配算法进行处理。
- （2）在中文中，由于偏正结构较多，所以从后向前进行匹配会提高精确度，因此，逆向最大匹配算法比正向最大匹配算法的误差要小。统计结果表明，单纯使用正向最大匹配的误差率为 $1/169$ ，单纯使用逆向最大匹配的误差率为 $1/245$ 。例如，对“你今天很好看”这一句文本进行分词，按照正向最大匹配算法得到的分词结果是“你 / 今天 / 很好 / 看”，按照逆向最大匹配算法得到的分词结果是“你 / 今天 / 很 / 好看”。

3.双向最大匹配算法 (BM) - Bi-directionction Matching method

- ◀ **理解**：把正向最大匹配与逆向最大匹配这两种算法都实施一遍，比较后选择最优结果。
- ◀ **原理**：将正向最大匹配法和逆向最大匹配法得到的分词结果进行比较，从而决定正确的分词方法。



3.双向最大匹配算法（BM）

- ◀ SunM.S. 和 Benjamin K.T.（1995）的研究表明，中文有 90.0%左右的句子用正向最大匹配算法和逆向最大匹配算法进行分词的结果是完全重合且正确的。如前面分析的例子，对于“秦皇岛今天晴空万里”这句文本，用正向最大匹配法和逆向最大匹配法进行中文分词的结果是完全一样且正确的。
- ◀ 除此之外，只有大概 9.0%的句子使用两种分词方法得到的结果不一样，但其中必有一个是正确的（歧义检测成功）；只有不到 1.0%的句子，或者正向最大匹配算法和逆向最大匹配算法的分词结果虽重合却是错的，或者正向最大匹配算法和逆向最大匹配算法分词结果不同但两个都不对（歧义检测失败）。鉴于此，双向最大匹配算法在实用中文信息处理系统中得以广泛使用。

3.双向最大匹配算法（BM）——算法举例

【例1】用双向最大匹配算法对“我一个人去海边玩”进行中文分词。

【案例分析】

根据前面的例子，我们可以得到如下结果：

正向匹配的分词结果：“我”“一个”“人”“去”“海边”“玩”。

逆向匹配的分词结果：“我”“一”“个人”“去”“海边”“玩”。

可以发现，两种算法的分词结果词数相同，虽然分词结果略有不同，但结果中单字数量相同，所以分词结果任取一个即可。

3.双向最大匹配算法（BM）——算法举例

【例2】用双向最大匹配算法对“孩子喜欢在野生动物园玩”进行中文分词。

【案例分析】

根据前面的例子，我们可以得到如下结果：

正向匹配的分词结果：“孩子”“喜欢”“在野”“生动”“物”“园”“玩”。

逆向匹配的分词结果：“孩子”“喜欢”“在”“野生”“动物园”“玩”。

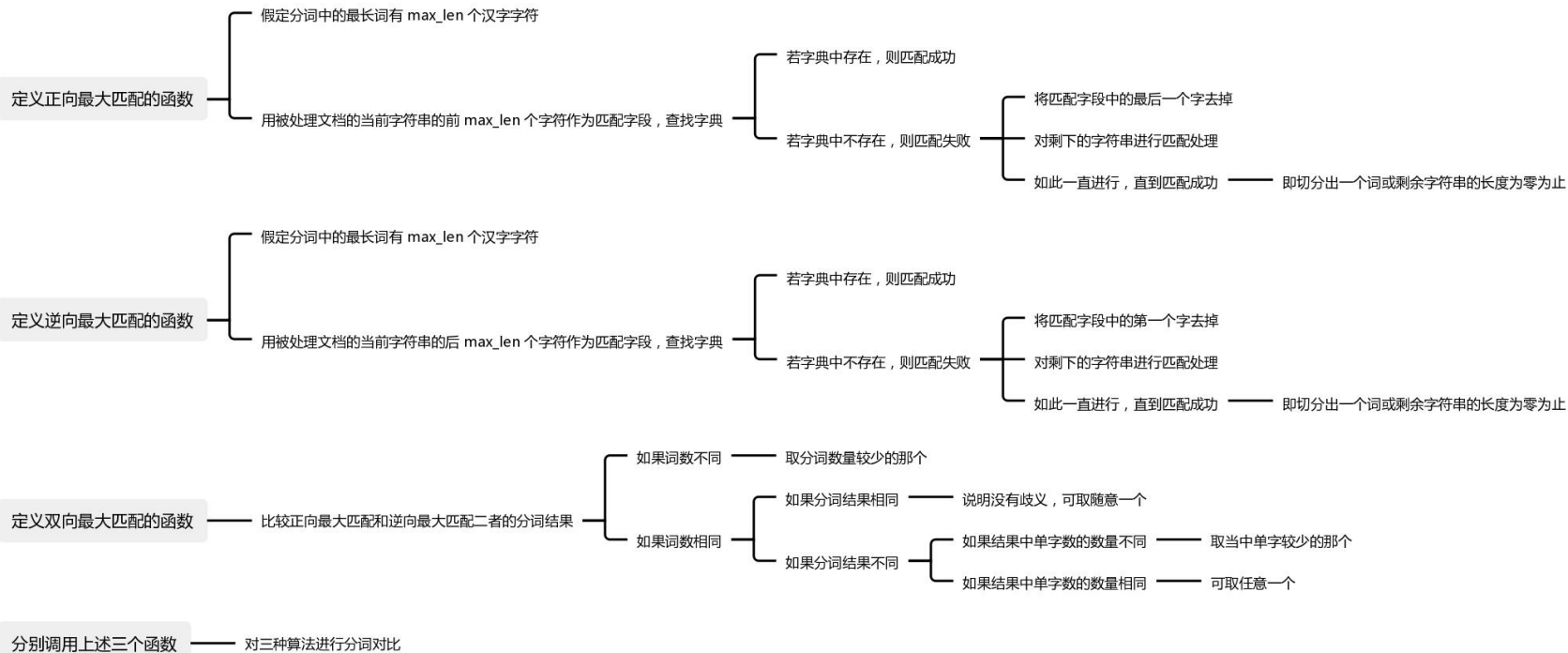
可以发现，两种算法的分词结果是不同的，结果中正向最大匹配的分词结果词数比逆向匹配的分词结果词数多，所以分词结果取词数较少的，即逆向最大匹配的分词结果。

◀ **注意事项：**基于词表的分词算法虽然简单快速，但对于未登录词以及切分歧义的情况无法处理。

4.基于词表的分词案例

【例】对“在野生动物园玩”这句中文文本，分别使用三种基于词表的分词算法进行分词。

案例分析



4.基于词表的分词案例——（1）正向最大匹配函数

```
def FMM(dict, sentence):          # 正向最大匹配算法FMM函数，参数dict: 词典，参数sentence: 句子
    fmmresult = []
    max_len = max([len(item) for item in dict]) # max_len定义为词典中最长词长度
    start = 0
    while start != len(sentence):    # FMM 为正向，start 从初始位置开始，指向结尾即为结束
        index = start + max_len      # index 的初始值为 start 的索引 + 词典中元素的最大长度或句子末尾
        if index > len(sentence):
            index = len(sentence)
        for i in range(max_len):
            # 当分词在字典中时或分到最后一个字时，将其加入到结果列表中
            if (sentence[start:index] in dict) or (len(sentence[start:index]) == 1):
                # print(sentence[start:index], end='/')
                fmmresult.append(sentence[start:index])
                start = index          # 分出一个词，start 设置到 index 处
                break
            index += -1                # 如果匹配失败，则去掉最后一个字符
    return fmmresult
```

1 正向最大匹配 FMM 动图演示 ①

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

FMM 函数（简化版）

```
1 def FMM(dict, sentence):
2     result = []
3     max_len = 4      # 词典中最长词长度
4     start = 0        # 从句首开始
5     while start < len(sentence):
6         index = start + max_len  # 初始取最长
7         if index > len(sentence):
8             index = len(sentence)
9         for i in range(max_len):
10            if (sentence[start:index] in dict) or \
11                (len(sentence[start:index]) == 1):
12                result.append(sentence[start:index])
13                start = index  # 分出一个词
14                break
15            index -= 1        # 匹配失败，去掉最后一个字
16     return result
```



词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子与扫描窗口



从位置 0 开始，取最长的 4 个字作为候选

当前候选词：

秦皇岛今



未匹配



去掉最后一个字，再试一次



当前变量状态

start = 0

index = 4

当前候选 = 秦皇岛今



FMM思路：从左到右，先找最长，再逐步缩短。

—— 如果当前候选不在词典中，就去掉最后一个字，继续尝试。

2 正向最大匹配 FMM 动图演示 ②

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

FMM 函数 (简化版)

```
1 def FMM(dict, sentence):
2     result = []
3     max_len = 4          # 词典中最长词长度
4     start = 0            # 从句首开始
5     while start < len(sentence):
6         index = start + max_len    # 初始取最长
7         if index > len(sentence):
8             index = len(sentence)
9         for i in range(max_len):
10            if (sentence[start:index] in dict) or \
11                (len(sentence[start:index]) == 1):
12                result.append(sentence[start:index]) # 加入结果
13                start = index          # 移动起始位置
14                break                  # 分出一个词
15            index -= 1                # 匹配失败, 去掉最后一个字
16     return result
```

📖 词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子与扫描窗口



当前候选词:

秦皇岛



匹配成功



当前变量状态

start = 3

index = 3

当前候选 = 秦皇岛

当前结果: [秦皇岛]

→ 加入结果列表, start 移动到下一个位置



FMM思路: 匹配成功, 就把这个词加入结果, 并继续向右扫描。 —— 不断重复这个过程, 直到句子扫描完毕。

3 正向最大匹配 FMM 动图演示 ③

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

FMM 函数 (简化版)

```
1 def FMM(dict, sentence):
2     result = []
3     max_len = 4          # 词典中最长词长度
4     start = 0            # 从句首开始
5     while start < len(sentence):
6         index = start + max_len    # 初始取最长
7         if index > len(sentence):
8             index = len(sentence)
9         for i in range(max_len):
10            if (sentence[start:index] in dict) or \
11                (len(sentence[start:index]) == 1):
12                result.append(sentence[start:index])
13                start = index    # 分出一个词
14                break
15         index -= 1          # 匹配失败，去掉最后一个字
16     return result
```

📖 词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子与扫描窗口

0	1	2	3	4	5	6	7	8
秦	皇	岛	今	天	晴	空	万	里

↑ 从位置 3 开始，再取最长的 4 个字作为候选

当前候选词：
今天晴空 × 未匹配

↓
去掉最后一个字，再试一次

📄 当前结果

当前结果：
[秦皇岛]

⚙️ 当前变量状态

start = 3

index = 7

当前候选 = 今天晴空



FMM思路：每到一个新位置，仍然先取最长候选。—— 如果当前候选不在词典中，就去掉最后一个字，继续尝试。

4 正向最大匹配 FMM 动图演示 ④

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

FMM 函数 (简化版)

```
1 def FMM(dict, sentence):
2     result = []
3     max_len = 4          # 词典中最长词长度
4     start = 0            # 从句首开始
5     while start < len(sentence):
6         index = start + max_len    # 初始取最长
7         if index > len(sentence):
8             index = len(sentence)
9         for i in range(max_len):
10            if (sentence[start:index] in dict) or \
11                (len(sentence[start:index]) == 1):
12                result.append(sentence[start:index])
13                start = index      # 移到下一个位置
14            break
15        index -= 1              # 匹配失败, 去掉最后一个字
16    return result
```

词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子与扫描窗口

0	1	2	3	4	5	6	7	8
秦	皇	岛	今	天	晴	空	万	里

从位置 3 开始, 取最长的 4 个字作为候选

当前候选词:

今天



匹配成功

当前结果:

[秦皇岛, 今天]

加入结果列表, 继续向右扫描



当前变量状态

start = 5

index = 5

当前候选 = 今天



FMM思路: 候选缩短后, 只要命中词典, 就立刻切分。—— 如果当前候选词在词典中, 就加入结果列表, 并从该位置继续向右扫描。

4 正向最大匹配 FMM 动图演示 ④

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

FMM 函数 (简化版)

```
1 def FMM(dict, sentence):
2     result = []
3     max_len = 4          # 词典中最长词长度
4     start = 0            # 从句首开始
5     while start < len(sentence):
6         index = start + max_len    # 初始取最长
7         if index > len(sentence):
8             index = len(sentence)
9         for i in range(max_len):
10            if (sentence[start:index] in dict) or \
11                (len(sentence[start:index]) == 1):
12                result.append(sentence[start:index])
13                start = index      # 分出一个词
14                break
15            index -= 1            # 匹配失败, 去掉最后一个字
16     return result
```

📖 词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子分词结果 (最终效果)

秦皇岛

今天

晴空万里

↑ 当前扫描位置 (3-4)

FMM 算法步骤回顾

1

先从左到右
扫描

2

每次先取
最长候选

3

不匹配就
去掉最后一个字

4

匹配成功
就加入结果
并继续

当前状态：

start = 5
index = 5
当前候选 = 今天

本步操作：

继续缩短候选，
变成「今天」

⚙️ 匹配结果：

当前候选词：今天 匹配成功

当前结果：[秦皇岛, 今天]



总结：FMM 本质上就是从左到右，不断尝试最长词，失败就缩短，成功就输出。

5 正向最大匹配 FMM 动图演示 ⑤

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

FMM 函数 (简化版)

```
1 def FMM(dict, sentence):
2     result = []
3     max_len = 4          # 词典中最长词长度
4     start = 0            # 从句首开始
5     while start < len(sentence):
6         index = start + max_len    # 初始取最长
7         if index > len(sentence):
8             index = len(sentence)
9         for i in range(max_len):
10            if (sentence[start:index] in dict) or \
11                (len(sentence[start:index]) == 1):
12                result.append(sentence[start:index])
13                start = index    # 分出一个词
14                break
15            index -= 1          # 匹配失败, 去掉最后一个字
16     return result
```

📖 词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子与扫描窗口

0	1	2	3	4	5	6	7	8
秦	皇	岛	今	天	晴	空	万	里

从位置 5 开始，↑ 取得最长的 4 个字作为候选

当前候选词：

晴空万里 ✓ 匹配成功

当前结果：

[秦皇岛, 今天, 晴空万里]

剩余部分正好匹配成功

⚙️ 当前变量状态

start = 9

index = 9

当前候选 = 晴空万里



FMM思路：如果最长候选本身就在词典中，就直接切分。

—— 这样可以直接得到一个完整的词，继续处理后续部分。

4.基于词表的分词案例——（2）逆向最大匹配函数

```
def RMM(dict, sentence):          # 逆向最大匹配算法RMM函数，参数dict: 词典，参数sentence: 句子
    rmmresult = []
    max_len = max([len(item) for item in dict]) # max_len定义为词典中最长词长度
    start = len(sentence)
    while start != 0:             # RMM 为逆向，start 从末尾位置开始，指向开头位置即为结束
        index = start - max_len    # 逆向时 index 的初始值为 start 的索引 - 词典中元素的最大长度或句子开头
        if index < 0:
            index = 0
        for i in range(max_len):
            # 当分词在字典中时或分到最后一个字时，将其加入到结果列表中
            if (sentence[index:start] in dict) or (len(sentence[index:start]) == 1):
                # print(sentence[index:start], end='/')
                rmmresult.insert(0, sentence[index:start])
                start = index        # 分出一个词，start 设置到 index 处
                break
            index += 1              # 如果匹配失败，则去掉最前面一个字符
    return rmmresult
```

1 逆向最大匹配 RMM 动图演示 ①

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

RMM 函数 (简化版)

```
1 def RMM(dict, sentence):
2     result = []
3     max_len = 4 # 词典中最长词长度
4     start = len(sentence) # RMM 为逆向, start 从末尾开始
5     while start != 0:
6         index = start - max_len # 逆向时, 初始 index = start - max_len
7         if index > 0:
8             index = 0
9             for i in range(max_len):
10                 if (sentence[index:start] in dict) or \
11                     (len(sentence[index:start]) == 1):
12                     result.insert(0, sentence[index:start])
13                     start = index # 匹配成功, start 移到 index
14                     break
15             index += 1 # 否则向右移动, 逐步缩短候选词
16     return result
```



词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子与扫描窗口



当前候选词:
晴空万里 ✓ 匹配成功

当前结果: [晴空万里]

当前变量状态

start = 5
index = 5
当前候选 = 晴空万里



RMM 思路：从右向左，先取最长，再逐步缩短。

逆向最大匹配算法从句子末尾开始，首先尝试匹配词典中最长的候选词，如果不匹配，则逐步缩短候选词长度。

2 逆向最大匹配 RMM 动图演示 ②

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

RMM 函数 (简化版)

```
1 def RMM(dict, sentence):
2     rmmresult = []
3     max_len = 4          # 词典中最长词长度
4     start = len(sentence) # RMM 为逆向, start 从末尾开始
5     while start != 0:
6         index = start - max_len # 逆向时, 初始 index = start - max_len
7         if index > 0:
8             index = 0
9         for i in range(max_len):
10            if (sentence[index:start] in dict) or \
11                (len(sentence[index:start]) == 1):
12                rmmresult.insert(0, sentence[index:start])
13                start = index # 匹配成功, start 移到 index 处
14                break
15            index += 1        # 如果匹配失败, 则去掉最前面一个字符
16     return rmmresult
```



RMM 思路：从右往左扫，不匹配就从左边去掉一个字。



词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子与扫描窗口



当前候选词：

皇岛今天



未匹配

匹配失败, index += 1,
去掉最前面的一个字

当前结果：

晴空万里



当前变量状态

start = 5

index = 1

当前候选 = 皇岛今天

逆向最大匹配算法保持右侧后缀不变，
当候选词不在词典中时，从左侧逐个去掉字符，缩短候选词长度。

3 逆向最大匹配 RMM 动图演示 ③

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

RMM 函数 (简化版)

```
1 def RMM(dict, sentence):
2     result = []
3     max_len = 4 # 词典中最长词长度
4     start = len(sentence) # RMM 为逆向, start 从末尾开始
5     while start != 0:
6         index = start - max_len # 逆向时, 初始 index = start - max_len
7         if index > 0:
8             index = 0
9         for i in range(max_len):
10             if (sentence[index:start] in dict) or \
11                 (len(sentence[index:start]) == 1):
12                 result.insert(0, sentence[index:start])
13                 start = index # 匹配成功, start 移到 index
14                 break
15             index += 1 # 否则向右移动, 逐步缩短候选词
16     return result
```

📖 词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子与扫描窗口

0	1	2	3	4	5	6	7	8
秦	皇	岛	今	天	晴	空	万	里

继续右移并缩短候选词，
最终变成：今天

当前候选词：

今天



匹配成功

⚙️ 当前变量状态

start = 3
index = 3
当前候选 = 今天

当前结果：[今天 , 晴空万里]

成功后插入到结果最前面

💡 小提示

先尝试：皇岛今天 (不在词典)
再尝试：岛今天 (不在词典)
最终匹配：今天 (在词典中)



RMM 思路：逆向匹配时，命中词典后要把词插到结果最前面。

逆向最大匹配算法从句子末尾开始，首先尝试匹配词典中最长的候选词，如果不匹配，则逐步缩短候选词长度。

4 逆向最大匹配 RMM 动图演示 ④

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

RMM 函数（简化版）

```
1 def RMM(dict, sentence):
2     result = []
3     max_len = 4 # 词典中最长词长度
4     start = len(sentence) # RMM 为逆向, start 从末尾开始
5     while start != 0:
6         index = start - max_len # 逆向时, 初始 index = start - max_len
7         if index < 0:
8             index = 0
9         for i in range(max_len):
10             if (sentence[index:start] in dict) or \
11                 (len(sentence[index:start]) == 1):
12                 result.insert(0, sentence[index:start])
13                 start = index # 匹配成功, start 移动到 index 处
14                 break
15             index += 1 # 否则向右移动, 逐步缩短候选词
16     return result
```

📖 词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子与扫描窗口



当前候选词：

秦皇岛



匹配成功

⚙️ 当前变量状态

start = 0
index = 0
当前候选 = 秦皇岛

当前结果：[秦皇岛, 今天, 晴空万里]

⚙️ 插入到结果最前面，并移动到句首

💡 小提示

此时 start = 0，表示已经处理到句首，算法结束。



RMM 思路：每次从右往左找最长词，命中后插到结果最前面，直到句首。



5 逆向最大匹配 RMM 动图演示 ⑤

分词原理 · 算法演示 · 一步一步看懂

例句：秦皇岛今天晴空万里

RMM 函数 (简化版)

```
1 def RMM(dict, sentence):
2     result = []
3     max_len = max([len(item) for item in dict])
4     start = len(sentence) # RMM 为逆向, start 从末尾开始
5     while start != 0:
6         index = start - max_len # 逆向时, 初始 index = start - max_len
7         if index < 0:
8             index = 0
9         for i in range(max_len):
10             if (sentence[index:start] in dict) or \
11                 (len(sentence[index:start]) == 1):
12                 result.insert(0, sentence[index:start])
13                 start = index # 匹配成功, start 移到 index 处
14                 break
15         index += 1 # 否则向右移动, 逐步缩短候选词
16     return result
```



词典：秦皇岛、岛、今天、天晴、晴空万里、万里

句子分词结果 (最终效果)

秦皇岛

今天

晴空万里

最终结果：[秦皇岛, 今天, 晴空万里]

RMM 分词过程回顾

1

从右向左
扫描

2

每次先取
最长候选

3

不匹配就去掉
最前面一个字

4

匹配成功就
插到结果最前面



RMM 与 FMM 的区别：

FMM 从左往右切，
RMM 从右往左切。



本例处理顺序：

晴空万里 → 今天 → 秦皇岛



总结：RMM 本质上就是从右到左，不断尝试最长词，失败就缩短左边，成功就输出。



4.基于词表的分词案例——（3）双向最大匹配函数

```
def BM(dict, sentence):          # 双向最大匹配（BM），参数dict: 词典，参数sentence: 句子
    res1 = FMM(dict, sentence)    # res1 为 FMM 结果
    res2 = RMM(dict, sentence)    # res2 为 RMM 结果
    if len(res1) == len(res2):    # 若分词数相同，则分情况讨论
        if res1 == res2:         # 若FMM 与 RMM 的结果相同，则可取任意一个
            return res1
        else:                    # res1_sn 和 res2_sn 为两个分词结果的单字数量，若二者不同，则返回单字较少的
            res1_sn = len([i for i in res1 if len(i) == 1])
            res2_sn = len([i for i in res2 if len(i) == 1])
            return res1 if res1_sn < res2_sn else res2
    else:                         # 若分词数不同，则取分出词较少的
        return res1 if len(res1) < len(res2) else res2
```

双向最大匹配 BM 动图演示 ①

例句： 研究生生命起源

词典： 研究生、研究、生命、命、起源

BM 函数（简化版）

```
def BM(dict, sentence):  
    res1 = FMM(dict, sentence)  
    res2 = RMM(dict, sentence)  
    if len(res1) == len(res2):  
        if res1 == res2:  
            return res1  
        else:  
            res1_sn = len([w for w in res1 if len(w) == 1])  
            res2_sn = len([w for w in res2 if len(w) == 1])  
            return res1 if res1_sn < res2_sn else res2  
    else:  
        return res1 if len(res1) < len(res2) else res2
```

BM 的整体流程

1 先用 FMM 做正向最大匹配

FMM 结果

[研究生, 命, 起源]

2 再用 RMM 做逆向最大匹配

RMM 结果

[研究, 生命, 起源]

3 比较两个结果

BM 会按照规则
比较这两个结果，
选出更合理的分词结果。



BM 思路： 先分别做 FMM 和 RMM，再按规则选更合理的结果。

BM 结合了正向最大匹配和逆向最大匹配，利用两种不同方向的分词结果进行比较，可以减少分词歧义，提高分词的准确性。



双向最大匹配 BM 动图演示 ②

例句： 研究生生命起源

词典： 研究生、研究、生命、命、起源

BM 函数（简化版）

```
def BM(dict, sentence):  
    res1 = FMM(dict, sentence)  
    res2 = RMM(dict, sentence)  
    if len(res1) == len(res2):  
        if res1 == res2:  
            return res1  
        else:  
            res1_sn = len([w for w in res1 if len(w) == 1])  
            res2_sn = len([w for w in res2 if len(w) == 1])  
            return res1 if res1_sn < res2_sn else res2  
    else:  
        return res1 if len(res1) < len(res2) else res2
```

① 先用 FMM 做正向最大匹配

从左向右扫描，每次取能匹配到的最长词

研 究 生 命 起 源

扫描方向：从左到右（正向）

第1步：

从开头取最长词
匹配到“研究生”

研究生

第2步：

剩余部分匹配
得到“命”

命

第3步：

继续匹配
得到“起源”

起源

FMM 结果

[研究生, 命, 起源]

单字词数 = 1

(其中“命”为单字词)



BM 思路：

BM 第一步：先做 FMM，得到正向最大匹配结果。



双向最大匹配 BM 动图演示 ③

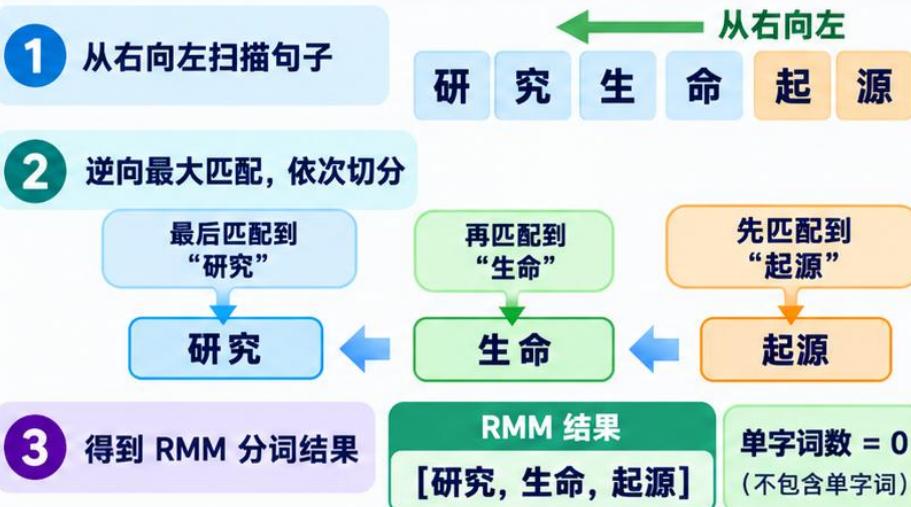
例句： 研究生生命起源

词典： 研究生、研究、生命、命、起源

BM 函数（简化版）

```
def BM(dict, sentence):  
    res1 = FMM(dict, sentence)  
    res2 = RMM(dict, sentence)  
    if len(res1) == len(res2):  
        if res1 == res2:  
            return res1  
        else:  
            res1_sn = len([w for w in res1 if len(w) == 1])  
            res2_sn = len([w for w in res2 if len(w) == 1])  
            return res1 if res1_sn < res2_sn else res2  
    else:  
        return res1 if len(res1) < len(res2) else res2
```

RMM 的分词过程（从右向左，逆向最大匹配）



BM 第二步： 再做 RMM，得到逆向最大匹配结果。

从右向左进行最大匹配，得到逆向最大匹配的分词结果。本例中 RMM 的结果为 [研究, 生命, 起源]，且不包含单字词。

双向最大匹配 BM 动图演示 ④

例句： 研究生生命起源

词典： 研究生、研究、生命、命、起源

BM 函数（简化版）

```
def BM(dict, sentence):  
    res1 = FMM(dict, sentence)  
    res2 = RMM(dict, sentence)  
    if len(res1) == len(res2):  
        if res1 == res2:  
            return res1  
        else:  
            res1_sn = len([w for w in res1 if len(w) == 1])  
            res2_sn = len([w for w in res2 if len(w) == 1])  
            return res1 if res1_sn < res2_sn else res2  
    else:  
        return res1 if len(res1) < len(res2) else res2
```

第三步：词数相同但结果不同，比较单字词数量



BM 第三步： 如果词数相同但结果不同，就比较单字词数量。

当 FMM 和 RMM 的分词词数相同时，如果两个结果不完全相同，就分别统计单字词的数量，选择单字词数量更少的结果。



双向最大匹配 BM 动图演示 ⑤

例句： 研究生生命起源

词典： 研究生、研究、生命、命、起源

BM 函数（简化版）

```
def BM(dict, sentence):  
    res1 = FMM(dict, sentence)  
    res2 = RMM(dict, sentence)  
    if len(res1) == len(res2):  
        if res1 == res2:  
            return res1  
        else:  
            res1_sn = len([w for w in res1 if len(w) == 1])  
            res2_sn = len([w for w in res2 if len(w) == 1])  
            return res1 if res1_sn < res2_sn else res2  
    else:  
        return res1 if len(res1) < len(res2) else res2
```

BM 的最体选择



BM 会按照以下规则进行选择：

- 1 若两个结果完全相同，直接返回；
- 2 若词数不同，返回词数更少的；
- 3 若词数相同但结果不同，返回单字词更少的。



BM 总结：

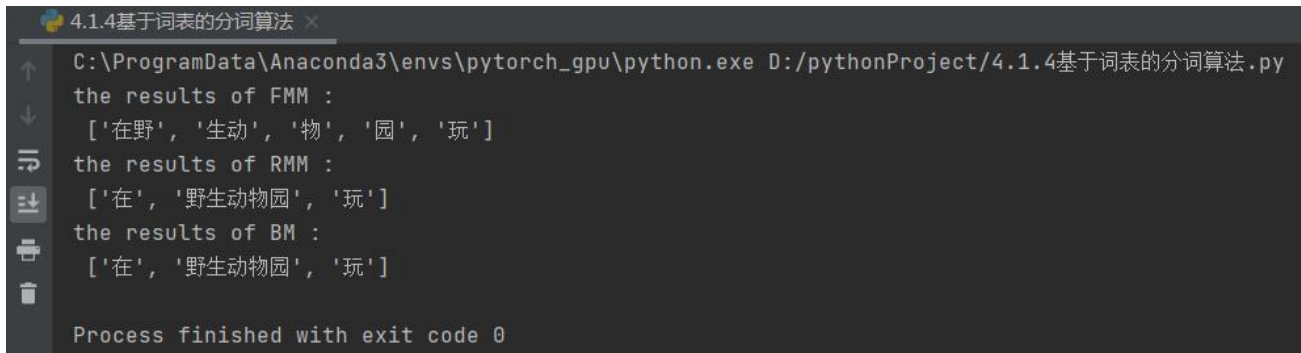
结合 FMM 和 RMM，选出更合理的分词结果。



4.基于词表的分词案例——（4）调用函数

```
dict = ['今日','阳光明媚','光明','明媚','阳光','我们','在','在野','生动','野生','动物园','野生动物园','物','园','玩']  
sentence = '在野生动物园玩'  
print("the results of FMM :\n", FMM(dict, sentence), end="\n") # 调用FMM函数，输出FMM分词结果  
print("the results of RMM :\n", RMM(dict, sentence), end="\n") # 调用RMM函数，输出RMM分词结果  
print("the results of BM :\n", BM(dict, sentence)) # 调用BM函数，输出BM分词结果
```

【运行结果】



```
4.1.4基于词表的分词算法 x  
C:\ProgramData\Anaconda3\envs\pytorch_gpu\python.exe D:/pythonProject/4.1.4基于词表的分词算法.py  
the results of FMM :  
['在野', '生动', '物', '园', '玩']  
the results of RMM :  
['在', '野生动物园', '玩']  
the results of BM :  
['在', '野生动物园', '玩']  
Process finished with exit code 0
```



02

基于统计模型的分词算法

1. N-gram模型

N-gram 模型称为 **N 元模型**，它是一种语言模型（Language Model, LM），该语言模型是一个基于概率的判别模型，其输入是一句话（词的顺序序列），输出是这句话的概率，即这句话里所有词的联合概率。

在分词中应用 N-gram 模型是将文本里的内容按照字节进行大小为 N 的滑动窗口操作，形成长度为 N 的字节片段序列。每一个字节片段称为 gram，对所有 gram 的出现频率进行统计，并且按照事先设定好的阈值进行过滤，形成关键 gram 列表，即该文本的向量特征空间，列表中的每一种 gram 就是一个特征向量维度。

N-gram 模型可应用在文化研究、分词应用、语音识别、输入法、词性标注、垃圾短信分类、机器翻译、语音识别、模糊匹配等领域。

2. N-gram模型分词算法——（1）基本思想

此算法假设每个词的出现只与它之前的 $N-1$ 个词相关，通过大量的语料统计便可以得知句子中每个词的出现概率，继而计算出整个句子的出现概率。如果一个句子的出现概率越大，则越符合自然语言的规律。

N 元模型认为当前词的出现概率只与它前面的 $N-1$ 个词有关。通常 N 可以取 1、2、3、4，其中 N 取 1、2、3 时分别称为 unigram（一元分词）、bigram（二元分词）、trigram（三元分词），最常用的是 bigram 和 trigram。

理论上， N 越大则 N -gram 模型越准确，也越复杂，所需计算量和训练语料数据量也越大。

2. N-gram模型分词算法——（2）概率计算

词出现的概率可以直接从语料中统计 N 个词同时出现的次数得到。对于一个句子W，假设 W 是由词序列 W1，W2，W3，.....,Wn 组成的，那么概率可按如下公式计算。

$$P(W) = P(W_1 W_2 W_3 \dots W_n) = P(W_1) P(W_2 | W_1) P(W_3 | W_1 W_2) \dots P(W_n | W_1 W_2 \dots W_{n-1})$$

当N=2时，当前词只依赖它前面的词（bigram）：

$$P(W) = P(W_1 W_2 W_3 \dots W_n) = P(W_1) P(W_2 | W_1) P(W_3 | W_2) \dots P(W_n | W_{n-1})$$

案例 3: N-gram 用“谁跟谁常一起出现”选切分方案

待分词：脑电图异常放电

方案 1

脑电图 / 异常放电 $P = 0.62$

方案 2

脑电 / 图异常 / 放电 $P = 0.08$

方案 3

脑 / 电图 / 异常 / 放电 $P = 0.19$

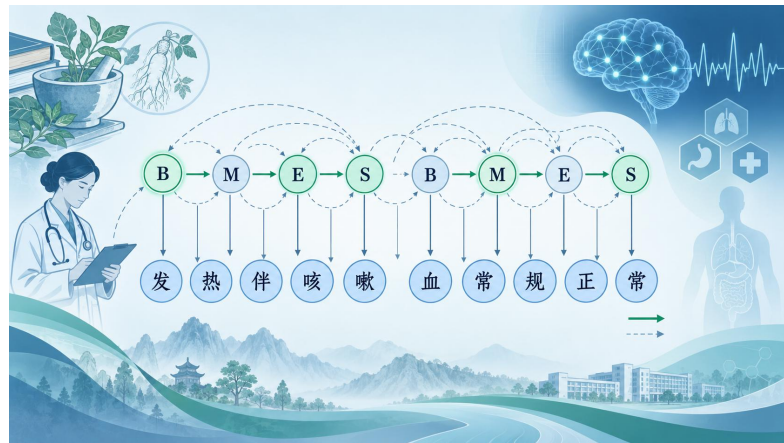


选择逻辑

整体概率最大，说明这种切法更像“自然语言里常见的说法”。



案例 4: HMM 把分词变成“给每个字贴标签”



句子：发热伴咳嗽，血常规正常



B

Begin, 词的开头,
如“发热”的“发”。

M / E / S

中间、结尾、单字成
词。模型要找最可能
的标签序列。

Viterbi 思路

每一步只保留当前最优路径，最后回溯得到整句
最优标签。

2. N-gram模型分词算法——（3）操作步骤

基于 N-gram 模型的分词算法的操作一般有如下步骤：

- ①建立 N-gram 统计语言模型。
- ②对句子进行单词划分，找出所有可能的分词情况。
- ③对分词的划分结果进行概率计算，找出出现可能性最大的分词序列。

2. N-gram模型分词算法——（4）算法举例（bigram）

【例】基于 N-gram 模型算法对“我喜欢观赏日出”进行中文分词。

① 待分词文本。

X = "我喜欢观赏日出"

② 对于待分词文本提出分词方案。

Y1 = {"我喜", "欢观", "赏", "日出"}

Y2 = {"我喜", "欢", "观赏", "日出"}

Y3 = {"我", "喜", "欢观", "赏日出"}

Y4 = {"我", "喜欢", "观赏", "日出"}

③ 概率计算。

$P(Y1) = P(\text{我喜}) * P(\text{欢观} | \text{我喜}) * P(\text{赏} | \text{欢观}) * P(\text{日出} | \text{赏})$

$P(Y2) = P(\text{我喜}) * P(\text{欢} | \text{我喜}) * P(\text{观赏} | \text{欢}) * P(\text{日出} | \text{观赏})$

$P(Y3) = P(\text{我}) * P(\text{喜} | \text{我}) * P(\text{欢观} | \text{喜}) * P(\text{赏日出} | \text{欢观})$

$P(Y4) = P(\text{我}) * P(\text{喜欢} | \text{我}) * P(\text{观赏} | \text{喜欢}) * P(\text{日出} | \text{观赏})$

案例分析：

在 4 个方案中，“我喜欢”在语料库中比较常见，所以 $P(\text{喜欢} | \text{我})$ 的概率就比较大。而“欢观赏日出”这样的词出现的可能性比较低，所以 $P(\text{赏日出} | \text{欢观})$ 的概率就比较低，同理分析其他概率，可得出 $P(Y4)$ 比 $P(Y1)$ 、 $P(Y2)$ 、 $P(Y3)$ 的值大，因此第四种分词方案最佳。

2. N-gram模型分词算法——（5）特点

基于 N-gram 模型的分词算法是在原有中文算法基础上进行了改进，设计并且实现了新的中文分词系统，既实现了文本的快速分词，又提高了中文分词的准确性，但其计算开销比较大，并且仍然存在未登录词难以处理的问题。

3. 基于统计模型的分词案例

【例】对“我喜欢观赏日出”使用N-gram模型的分词算法进行分词。

【案例分析】此分词算法分两步开展：

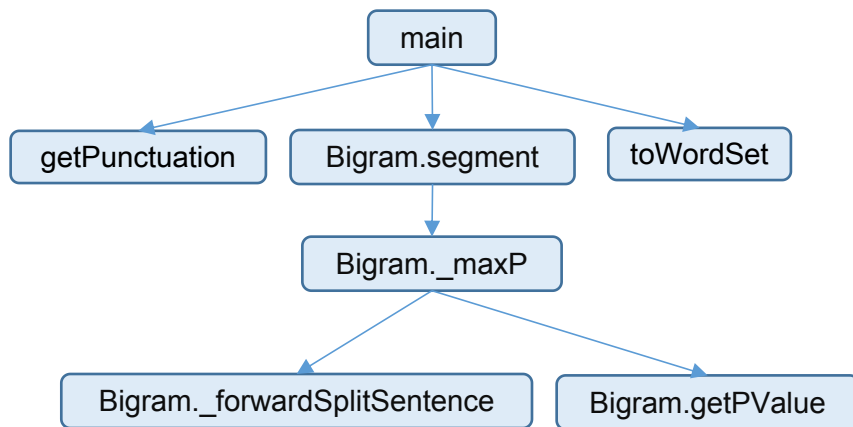
- (1) 获取待切分句子的所有切分方案；
- (2) 从所有方案中选取最优的切分方案。

依据封装原理，定义 Bigram 类进行实现，**Bigram**类中包含有：

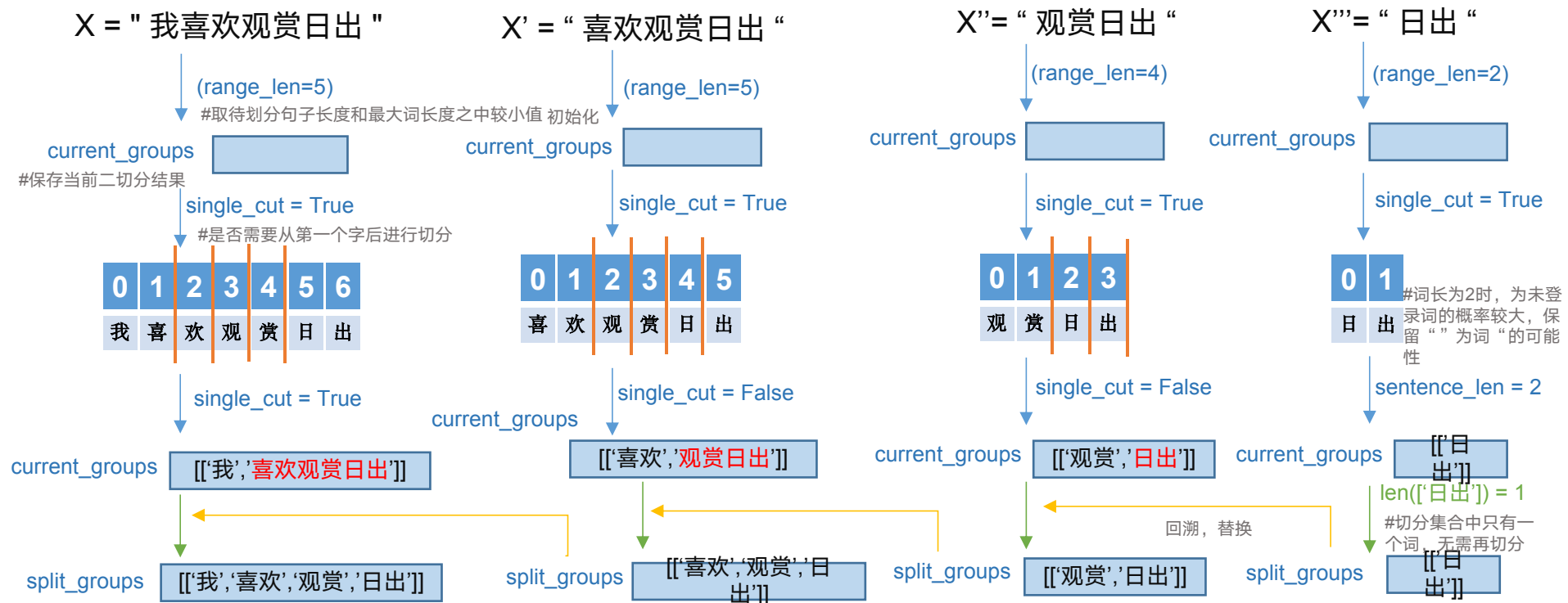
- (1) `__init__`函数进行初始化
- (2) `_forwardSplitSentence`函数用于获取所有可能的切分方案
- (3) `getPValue`函数用于查询二元概率
- (4) `_maxP`函数用于计算最大概率的切分组合
- (5) `segment`函数进行分词调用

其他重要函数还有：

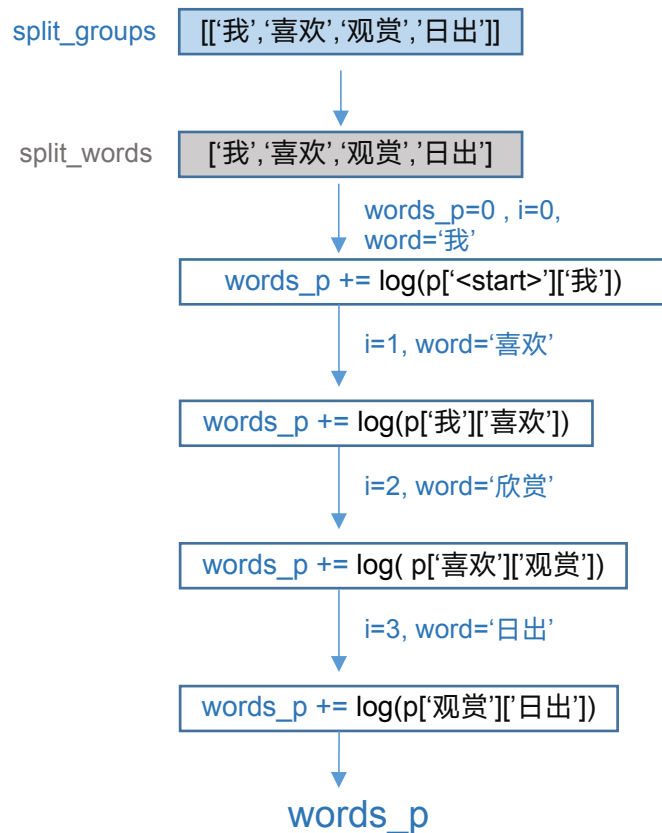
`getPunctuation`函数用于获取标点，部分标点可以认为是自然分割符
`toWordSet`函数用于获取词典



3. 基于统计模型的分词案例—— (2) `_forwardSplitSentence`函数用于获取所有可能的切分方案



3. 基于统计模型的分词案例—— (4) `_maxP`函数用于计算最大概率的切分组合



利用log函数可以巧妙将乘法转换成加法运算
第一个词没有前向依赖词，所以设置<start>

在词表中依次查询二元概率进行累加
由于是Bigram模型，所以当前词出现的概率只与前一个词出现的概率相关



03

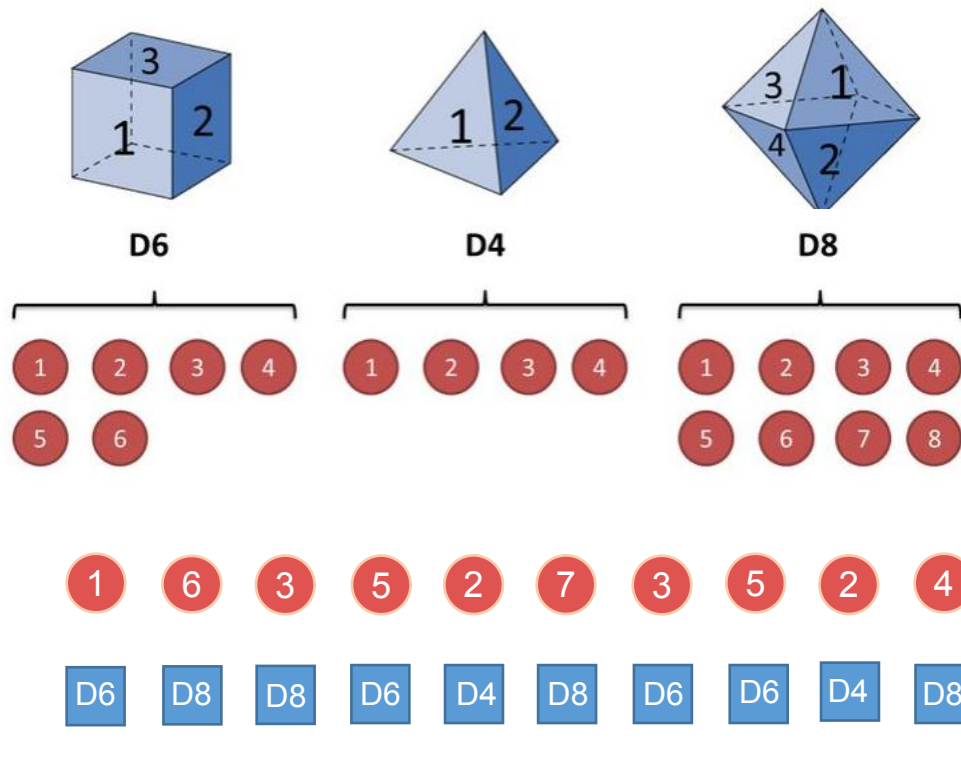
基于序列标注的分词算法

1. 隐马尔可夫模型——（1）概述

隐马尔可夫模型（Hidden Markov Model, HMM）是**关于时序的概率模型**，描述由一个隐藏的马尔可夫链随机生成不可观测（或称为隐状态）的状态随机序列，再由各个状态生成一个观测从而产生观测随机序列的过程。隐藏的马尔可夫链随机生成的状态的序列，称为状态序列；每个状态生成一个观测，而由此产生的观测的随机序列，称为观测序列。序列的每一个位置可以看作是一个时刻。基于序列标注的分词算法除了有基于隐马尔可夫模型，还有基于 CRF、基于 LSTM 等多种分词算法。

隐马尔可夫模型常应用于序列标注的问题。它用于标注时，状态对应着标记，标注问题是给定观测序列预测其对应的标记序列。

1. 隐马尔可夫模型——（1）概述



2. 基于隐马尔可夫模型进行中文分词——（1）基本原理

◀ **基本原理**：HMM 模型把分词问题转化为序列标注问题，也就是给定一个句子作为输入，以“BEMS”组成的序列串作为输出，然后再进行分词，从而得到输入句子的划分。

◀ 其中：

- B（Begin）代表词的起始位置，
- M（Middle）代表词的中间位置，
- E（End）表示词的结束位置，
- S（Single）代表单字成词。

2. 基于隐马尔可夫模型进行中文分词——（2）算法举例

【例】基于 HMM，对“今天晴天，我们去浅水湾玩吧！”这句文本进行中文分词，见下表。

可见:	观测序列	今	天	晴	天	,	我	们	去	浅	水	湾	玩	吧	!
隐含:	状态序列	B	E	B	E	S	B	E	S	B	M	E	S	S	S

【案例分析】：原句：今天晴天，我们去浅水湾玩吧！

分词结果：今天 / 晴天 / , / 我们 / 去 / 浅水湾 / 玩 / 吧 / !

2. 基于隐马尔可夫模型进行中文分词——（3）对中文分词进行形式化描述

◀：设观测状态集合（输入句子序列）为 $O=\{o_1, o_2, \dots, o_n\}$ ，隐藏状态集合（“BMES”序列） $I=\{i_1, i_2, \dots, i_n\}$ ，中文分词就是对给定的观测序列，求解对应的最有可能的隐藏状态序列，即求解最大条件概率 $\max P(i_1, \dots, i_n \mid o_1, \dots, o_n)$ ，利用贝叶斯公式可得：

$$P(i_1, \dots, i_n \mid o_1, \dots, o_n) = P(o_1, \dots, o_n \mid i_1, \dots, i_n) P(i_1, \dots, i_n)$$

2. 基于隐马尔可夫模型进行中文分词——（3）对中文分词进行形式化描述

分词过程：

（1）初始状态概率分布 π ：即句子的第一个字属于 {B、E、M、S} 这四种状态的概率。根据四种状态的概率分析，句子第一个字状态为 E、M 的概率为 0，即句子开头第一个字只可能是词的开始或单字。

（2）状态转移概率矩阵 A：根据 HMM 基本假设中的马尔科夫假设，可以将问题简化为状态转移概率矩阵 A 就是从 一个状态转移到另一个状态所产生的概率矩阵，即一个 4×4 的二维矩阵，其中矩阵的坐标为 {B、E、M、S} $\times$ {B、E、M、S}。

（3）观测矩阵 B：根据观测独立性假设，观察值只取决于当前状态值，就可以认为观测矩阵 B 是在某个状态下的观察值的概率所形成的二维矩阵。比如，对于句子“今天晴天，我们去浅水湾玩吧！”， $B[0][0]$ 表示在状态“B”的条件下，出现“今”字的概率。

得到 HMM 以上信息后，就可以根据 Viterbi 算法完成分词了。

3. 维特比（Viterbi）算法——（1）基本原理

在 HMM 中，常用维特比算法。

- 基本原理：维特比（Viterbi）算法实际是用动态规划解马尔可夫模型预测问题，即用动态规划求概率最大路径（最优路径）。
- 它用于寻找最有可能产生观测事件序列的维特比路径——隐含状态序列。

3. 维特比 (Viterbi) 算法——(3) 算法举例

🔊【例】：基于典型的“盒子和球模型”进行抽球。假设有 3 个盒子，里面装有黑白颜色的球。球的颜色序列对应观测序列 O ，盒子的序列对应状态序列 S 。

已知初始状态概率分布 π (表 4-4)，状态转移矩阵 A (表 4-5)，状态观测矩阵 B (表 4-6) 分别为：

表 4-4 初始状态概率分布 π

S	π
1	0.3
2	0.5
3	0.2

表 4-5 状态转移矩阵 A

S/S	1	2	3
1	0.5	0.3	0.3
2	0.2	0.5	0.4
3	0.3	0.2	0.3

表 4-6 状态观测矩阵 B

S/O	白	黑
1	0.5	0.5
2	0.6	0.4
3	0.3	0.7

🔊观测序列为 $O=\{\text{白、黑、白}\}$ ，求状态序列 S ，其中 (1, 2, 3) 为状态。

3. 维特比 (Viterbi) 算法——(3) 算法举例

【案例分析】首先定义两个矩阵：

- $\text{weight}[3][3]$ ——表示在 t 时刻状态为 i 的所有单个路径中的概率最大值。
- $\text{path}[3][3]$ ——表示在 t 时刻状态为 i 的所有路径中概率最大的那条，它的第 $t-1$ 个节点是什么。

(1) 初始化。 $t = 1$ 时的“白”，分别是状态 1, 2, 3 的条件下观察得来的概率，计算如下：

	白
1	$0.3 \times 0.5 = 0.15$
2	$0.5 \times 0.6 = 0.3$
3	$0.2 \times 0.3 = 0.06$

此时， path 矩阵的第一列为 0。

3. 维特比 (Viterbi) 算法——(3) 算法举例

(2) 递归。根据 Viterbi 算法, $t = 2$ 时的“黑”, 是在前一时刻的状态下观察得来的, 因此, 先计算此时刻的状态最有可能是由前一时刻的哪个状态转换而来的, 取最大值, 再乘以前一时刻的状态下观察到“黑”的概率。因此可以得到 $\text{weight}[3][3]$ 矩阵、 $\text{path}[3][3]$ 矩阵如下:

表 4-8 $\text{weight}[3][3]$ 矩阵

	$t=1$	$t=2$	$t=3$
	白	黑	白
1	$0.3 \times 0.5 = 0.15$	$(0.15 \times 0.5) \times 0.5 = 0.0375$ $(0.3 \times 0.2) \times 0.5 = 0.03$ $(0.06 \times 0.3) \times 0.5 = 0.009$ Max = 0.0375	$(0.0375 \times 0.5) \times 0.5 = 0.009375$ $(0.06 \times 0.2) \times 0.5 = 0.006$ $(0.084 \times 0.3) \times 0.5 = 0.00126$ Max = 0.009375
2	$0.5 \times 0.6 = 0.3$	$(0.15 \times 0.3) \times 0.4 = 0.018$ $(0.3 \times 0.5) \times 0.4 = 0.06$ $(0.06 \times 0.2) \times 0.4 = 0.0048$ Max = 0.06	$(0.0375 \times 0.3) \times 0.6 = 0.00675$ $(0.06 \times 0.5) \times 0.6 = 0.018$ $(0.084 \times 0.2) \times 0.6 = 0.01008$ Max = 0.018
3	$0.2 \times 0.3 = 0.06$	$(0.15 \times 0.3) \times 0.7 = 0.0315$ $(0.3 \times 0.4) \times 0.7 = 0.084$ $(0.06 \times 0.3) \times 0.7 = 0.0126$ Max = 0.084	$(0.0375 \times 0.3) \times 0.3 = 0.003375$ $(0.06 \times 0.4) \times 0.3 = 0.0072$ $(0.084 \times 0.3) \times 0.3 = 0.00756$ Max = 0.00756

表 4-9 $\text{path}[3][3]$ 矩阵

	$t=1$	$t=2$	$t=3$
	白	黑	白
1	0	1	1
2	0	2	2
3	0	2	3

状态转移矩阵A中:

$t=1$ 时状态1, 转移到 $t=2$ 时状态1的概率=0.5

$t=1$ 时状态2, 转移到 $t=2$ 时状态1的概率=0.2

$t=1$ 时状态3, 转移到 $t=2$ 时状态1的概率=0.3

状态观测矩阵B中:

$t=2$ 时状态1, 黑球概率=0.5

$t=2$ 时状态1, 黑球概率=0.5

$t=2$ 时状态1, 黑球概率=0.5

3. 维特比 (Viterbi) 算法——(3) 算法举例

(3) 终止。在 $t = 3$ 时，最大概率为 0.018，对应的最优路径的终点为 2。

(4) 回溯。从最优路径的终点 2 开始，向前回溯，找到最优路径为：(2, 2, 2)。

5. 基于序列标注的分词案例

【例】对“我喜欢观赏日出”这句中文文本，使用基于 HMM 模型的分词算法进行分词。

【案例分析】

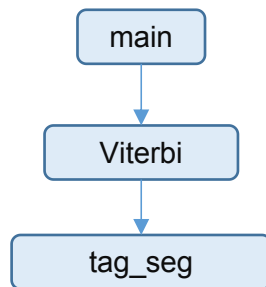
此分词算法分两步开展：

- (1) 获取切分句子的最优状态序列；
- (2) 将最优状态序列转换为切分方案。

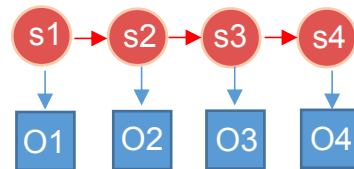
重要函数有：

Viterbi函数应用维特比算法来求解最优状态序列。

tag_seg函数根据状态序列进行分词。



状态链: BEMS标注序列



观测链: 待分词文本

当前的状态只取决于上一个状态
当前的观测值只取决于当前的状态值

5. 基于序列标注的分词案例——viterbi函数

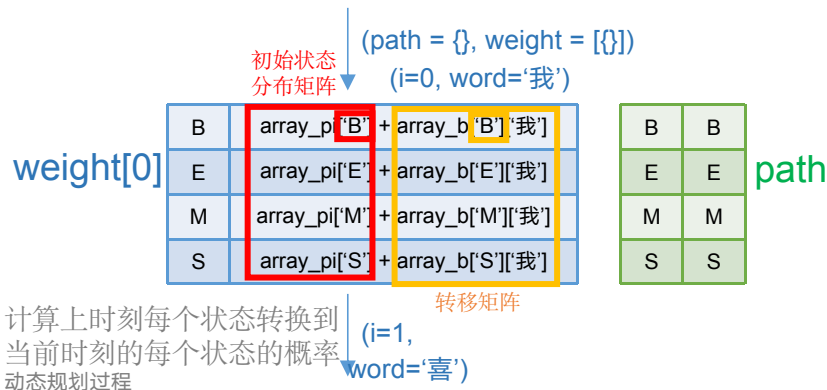
首先获取三元素：

1. 初始状态概率分布矩阵array_pi
2. 观测状态概率矩阵array_b
3. 状态转移概率矩阵array_a

两个重要变量：

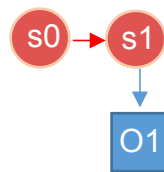
- $weight[t][state]$ 表示时刻t到达state状态的所有路径中，概率最大路径的概率值
- $path[state]$ 表示当前到达状态state所经过的最优路径

X = " 我喜欢观赏日出 "



5. 基于序列标注的分词案例——viterbi函数

i=0/ i=1	B	E	M	S
B	weight[0]['B'] + array_a['B']['B'] + array_b['B']['喜']	weight[0]['E'] + array_a['E']['B'] + array_b['B']['喜']	weight[0]['M'] + array_a['M']['B'] + array_b['B']['喜']	weight[0]['S'] + array_a['S']['B'] + array_b['B']['喜']
E	weight[0]['B'] + array_a['B']['E'] + array_b['E']['喜']	weight[0]['E'] + array_a['E']['E'] + array_b['E']['喜']	weight[0]['M'] + array_a['B']['E'] + array_b['E']['喜']	weight[0]['S'] + array_a['S']['E'] + array_b['E']['喜']
M	weight[0]['B'] + array_a['B']['M'] + array_b['M']['喜']	weight[0]['E'] + array_a['E']['M'] + array_b['M']['喜']	weight[0]['M'] + array_a['M']['M'] + array_b['M']['喜']	weight[0]['S'] + array_a['S']['M'] + array_b['M']['喜']
S	weight[0]['B'] + array_a['B']['S'] + array_b['S']['喜']	weight[0]['E'] + array_a['E']['S'] + array_b['S']['喜']	weight[0]['M'] + array_a['M']['S'] + array_b['S']['喜']	weight[0]['S'] + array_a['S']['S'] + array_b['S']['喜']



X = " 我喜欢观赏日出 "

(path = {}, weight = [])
(i=0, word='我')

weight[0]

B	array_pi['B'] + array_b['B']['我']
E	array_pi['E'] + array_b['E']['我']
M	array_pi['M'] + array_b['M']['我']
S	array_pi['S'] + array_b['S']['我']

B	B
E	E
M	M
S	S

path

计算上时刻每个状态转换到
当前时刻的每个状态的概率

(i=1,
word='喜')

weight[1]

B	-14.056415039093764,
E	-15.541616801202998
M	-15.109248798103819
S	-16.960517710360023

B	[S,B]
E	[B,E]
M	[B,M]
S	[S,S]

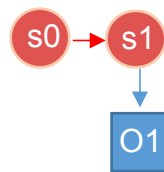
path

5. 基于序列标注的分词案例——viterbi函数

首先获取三元素：

1. 初始状态概率分布矩阵array Pi

i=1/ i=2	B	E	M	S
B	weight[1]['B'] + array_a['B']['B'] + array_b['B']['欢']	weight[1]['E'] + array_a['E']['B'] + array_b['B']['欢']	weight[1]['M'] + array_a['M']['B'] + array_b['B']['欢']	weight[1]['S'] + array_a['S']['B'] + array_b['B']['欢']
E	weight[1]['B'] + array_a['B']['E'] + array_b['E']['欢']	weight[1]['E'] + array_a['E']['E'] + array_b['E']['欢']	weight[1]['B'] + array_a['B']['E'] + array_b['E']['欢']	weight[1]['S'] + array_a['S']['E'] + array_b['E']['欢']
M	weight[1]['B'] + array_a['B']['M'] + array_b['M']['欢']	weight[1]['E'] + array_a['E']['M'] + array_b['M']['欢']	weight[1]['M'] + array_a['M']['M'] + array_b['M']['欢']	weight[1]['S'] + array_a['S']['M'] + array_b['M']['欢']
S	weight[1]['B'] + array_a['B']['S'] + array_b['S']['欢']	weight[1]['E'] + array_a['E']['S'] + array_b['S']['欢']	weight[1]['M'] + array_a['M']['S'] + array_b['S']['欢']	weight[1]['S'] + array_a['S']['S'] + array_b['S']['欢']



X = " 我喜欢观赏日出 "

(path = {}, weight = [{}])
(i=0, word='我')

B	array_pi['B'] + array_b['B']['我']
E	array_pi['E'] + array_b['E']['我']
M	array_pi['M'] + array_b['M']['我']
S	array_pi['S'] + array_b['S']['我']

B	B
E	E
M	M
S	S

path

计算上时刻每个状态转换到
当前时刻的每个状态的概率(i=1,
word='喜')

B	-14.056415039093764,
E	-15.541616801202998
M	-15.109248798103819
S	-16.960517710360023

B	[S,B]
E	[B,E]
M	[B,M]
S	[S,S]

path

(i=2, word='欢')

B	-23.563464488535256
E	-22.164436780888078
M	-25.312445154356144
S	-29.246163045326497

B	['B', 'E', 'B']
E	['S', 'B', 'E']
M	['S', 'B', 'M']
S	['B', 'E', 'S']

5. 基于序列标注的分词案例——viterbi函数

首先获取三元素：

1. 初始状态概率分布矩阵array_Pi
2. 观测状态概率矩阵array_b
3. 状态转移概率矩阵array_a

两个重要变量：

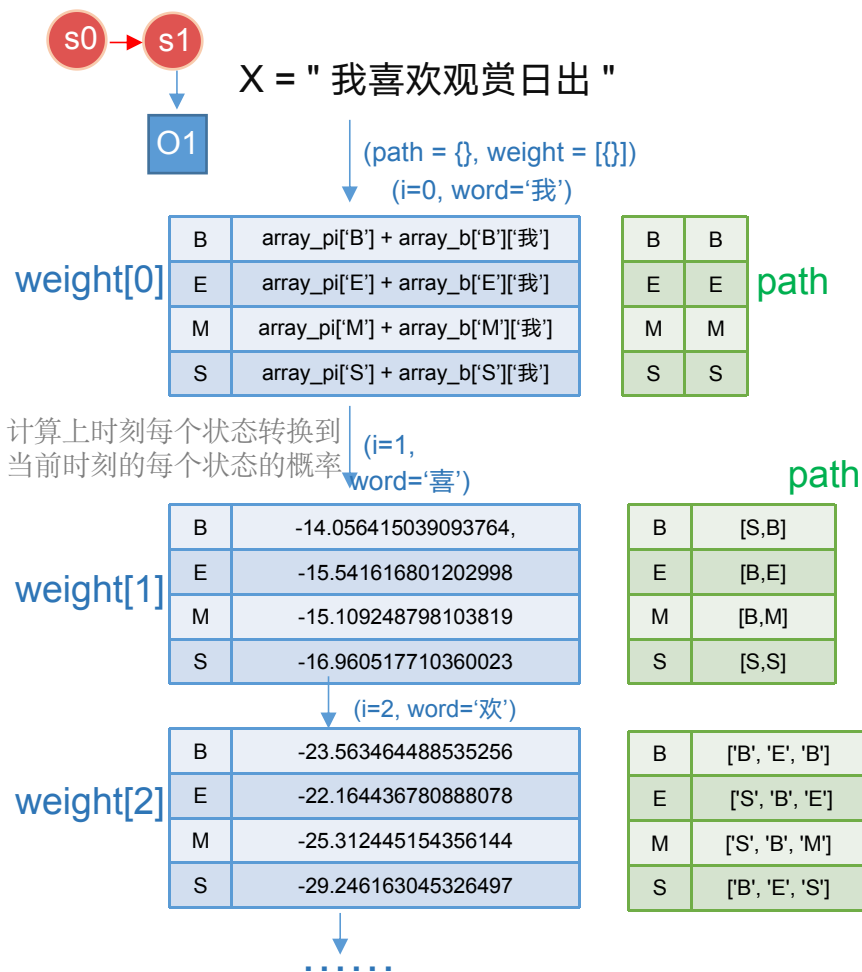
- $weight[t][state]$ 表示时刻t到达state状态的所有路径中，概率最大路径的概率值
- $path[state]$ 表示当前到达状态state所经过的最优路径

weight[6]

B	-53.16862134714856
E	-50.89845344774695
M	-53.75914262323005
S	-54.76447815700497

B	['S', 'B', 'E', 'B', 'E', 'S', 'B']
E	['S', 'B', 'E', 'B', 'E', 'B', 'E']
M	['S', 'B', 'E', 'B', 'E', 'B', 'M']
S	['S', 'B', 'E', 'B', 'E', 'S', 'S']

中文分词



5. 基于序列标注的分词案例——viterbi函数

i=0/ i=1	B		E		M		S	
i=1/ i=2	B		E		M		S	
B	weight[1]['B'] + array_a['B']['B'] + array_b['B']['欢']		weight[1]['E'] + array_a['E']['B'] + array_b['B']['欢']		weight[1]['M'] + array_a['M']['B'] + array_b['B']['欢']		weight[1]['S'] + array_a['S']['B'] + array_b['B']['欢']	
E	weight[1]['B'] + array_a['B']['E'] + array_b['E']['欢']		weight[1]['E'] + array_a['E']['E'] + array_b['E']['欢']		weight[1]['B'] + array_a['B']['E'] + array_b['E']['欢']		weight[1]['S'] + array_a['S']['E'] + array_b['E']['欢']	
M	weight[1]['B'] + array_a['B']['M'] + array_b['M']['欢']		weight[1]['E'] + array_a['E']['M'] + array_b['M']['欢']		weight[1]['M'] + array_a['M']['M'] + array_b['M']['欢']		weight[1]['S'] + array_a['S']['M'] + array_b['M']['欢']	
S	B	-53.16862134714856			B	['S', 'B', 'E', 'B', 'E', 'S', 'B']		
	E	-50.89845344774695			E	['S', 'B', 'E', 'B', 'E', 'B', 'E']		
	M	-53.75914262323005			M	['S', 'B', 'E', 'B', 'E', 'B', 'M']		
	S	-54.76447815700497			S	['S', 'B', 'E', 'B', 'E', 'S', 'S']		

中文分词

viterbi算法认为出现概率最大的序列为最优序列，因此选择weight[6]中最大的值为最优状态

s0

s1

取出第一个字“我”，把每个状态的初始概率与该状态转移到“我”字的概率相加，填入到weight[0]中，表示在0时刻达到每个状态的概率分布矩阵

(path = {}, weight = {})

(i=0, word='我')

记录达到当前每个状态所经过的最优路径

由于只有一个时刻，所以右侧值与左侧值相同

weight[0]

“我”字的概率

B	$\text{array_pi}['B'] + \text{array_b}['B']['\text{我}']$
E	$\text{array_pi}['E'] + \text{array_b}['E']['\text{我}']$
M	$\text{array_pi}['M'] + \text{array_b}['M']['\text{我}']$
S	$\text{array_pi}['S'] + \text{array_b}['S']['\text{我}']$

转移矩阵

(i=1, word='喜')

当i=1，截取第二个字“喜”时

path

B	[S,B]
E	[B,E]
M	[B,M]
S	[S,S]

weight[1]

动态规划过程

B	-14.056415039093764,
E	-15.541616801202998
M	-15.109248798103819
S	-16.960517710360023

weight[2]

(i=2, word='欢')

B	-23.563464488535256
E	-22.164436780888078
M	-25.312445154356144
S	-29.246163045326497

weight[6]

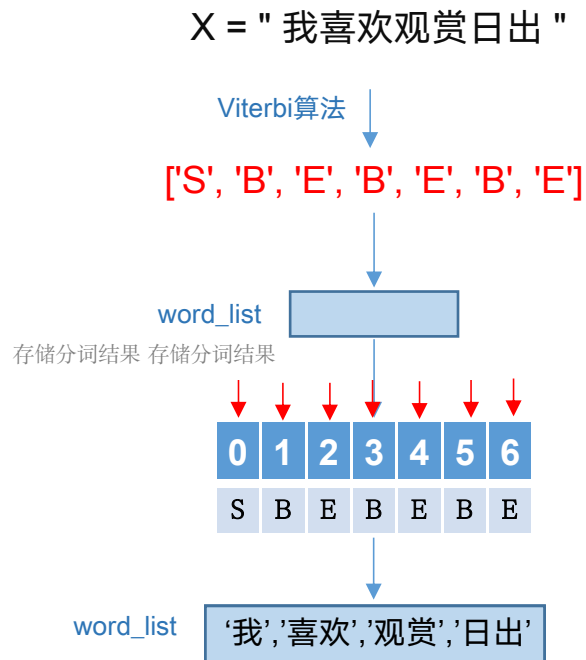
i=2 B状态时，i=1 E状态最优，向前回溯，寻最优路径：
i=1 E状态时最优为i=0 B状态

X = " 我喜欢观赏日出 "

.....

5. 基于序列标注的分词案例——tag_seg函数

将状态序列转换成分词结果





04

中文分词工具

1.常见的中文分词工具

HanLP 分词器、Jieba（结巴）分词、哈工大的语言技术平台 LTP 及其语言云 LTP-Cloud、清华大学的中文词法分析工具包 THULAC、北京大学的中文分词工具包 pkuseg、斯坦福分词器、基于深度学习的分词系统 KCWS、新加坡科技设计大学的中文分词器 ZPar、IKAnalyzer、Jcseg、复旦大学的 FudanNLP、中文文本处理库 SnowNLP、ansj 分词器、自然语言处理工具包 NLTK、玻森中文语义开放平台 BosonNLP、简易中文分词系统 SCWS、IKAnalyzer、庖丁解牛、中科院计算所 NLPIR 分词系统、腾讯文智、百度 NLP、阿里云 NLP、新浪云、搜狗分词、盘古分词等等。

Jieba（结巴）分词工具

2.Jieba 分词——（1）概述

- ◀ Jieba 分词是一个 Python 中文分词组件。
- ◀ **功能**：可以对中文文本进行分词、词性标注、关键词抽取等，并且支持自定义词典。
- ◀ **原理**：算法使用了基于前缀词典实现高效的词图扫描，生成句子中汉字所有可能生成词情况所构成的有向无环图，再采用动态规划查找最大概率路径，从而找出基于词频的最大切分组合。对于未登录词，它采用了基于汉字成词能力的 HMM 模型，使用了 Viterbi 算法。
- ◀ **特点**：在词典文件添加自定义词典速度快，适用于词典数量大于五千万。但是其自定义词典时，带空格的词并不支持。

2.Jieba 分词——（2）安装

全自动安装

在终端中输入如下命令进行安装：

`pip install jieba`，
或者 `pip3 install jieba`，
或者 `easy_install jieba`。

注：对 Python 2.X、Python3.X 均兼容

1

2

半自动安装

从官网 <http://pypi.python.org/pypi/jieba/> 下载，
下载后解压后运行 `python setup.py`。
或从 <https://github.com/fxsjy/jieba/tree/jieba3k>
下载。

手动安装

将已有的 jieba 目录放置于当前目录，
或者 site-packages 目录。

注：[site-packages](#)里放的是第三方包。
自动安装时，安装包会把对应的文件释
放到site-packages，手动时同理，把
jieba包复制到site-packages即可。

3

4

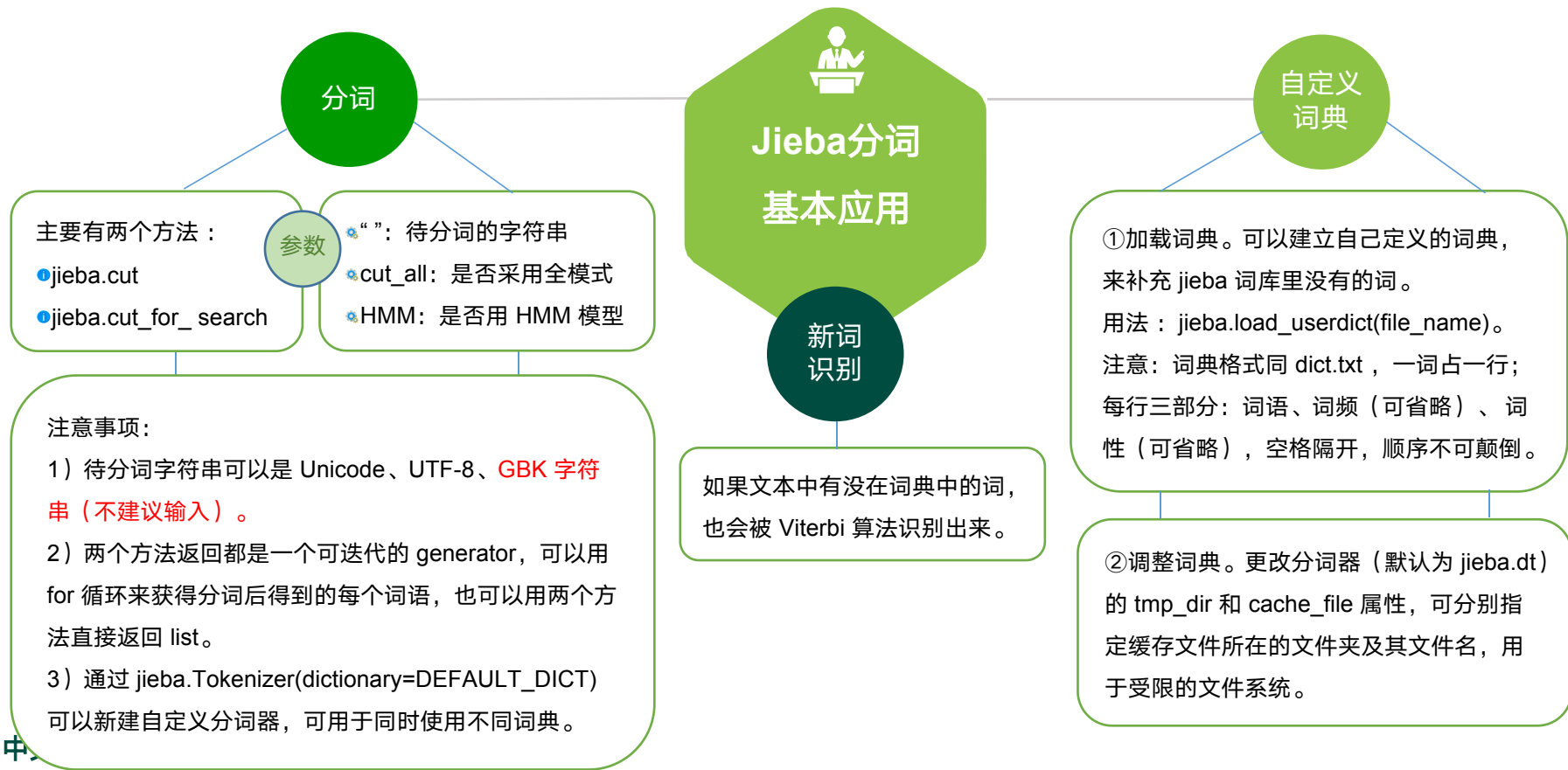
在PyCharm中安装

File——Setting，左侧Project: 工程名 ——
Project Interpreter，点击右侧“+”，在弹出
界面的搜索栏中输入“jieba”，点击 Install
Package 按钮。

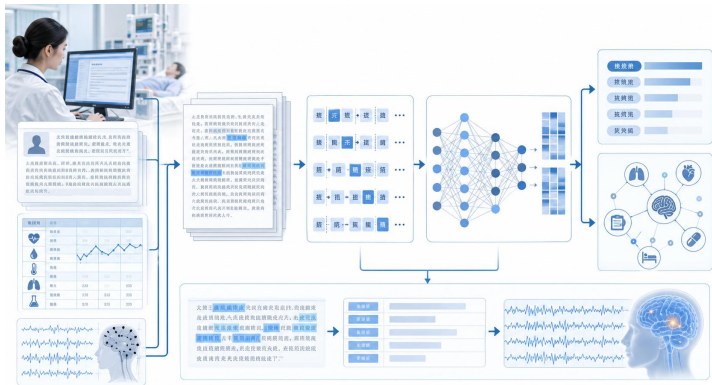
2.Jieba 分词——（3）三种分词模式



2.Jieba 分词——（4）基本应用



案例 2: Jieba 加医学自定义词典



没有医学词典

脑 / 电 / 图 / 左
/ 颞区



加入医学词典

脑电图 / 左颞区
/ 异常放电

代码提示

```
import jieba
jieba.load_userdict("medical_dict.txt")
jieba.add_word("左颞区",
freq=20000, tag="nz")
jieba.cut("脑电图提示左颞区异常放电")
```

Jieba 还会结合 HMM 和 Viterbi 处理未登录词。

3.Jieba 分词案例

【例】对“深圳技术大学是广东省和深圳市高标准建设的国际化、高水平、示范性一流应用技术大学”这句中文文本，使用 Jieba 分词工具进行分词。

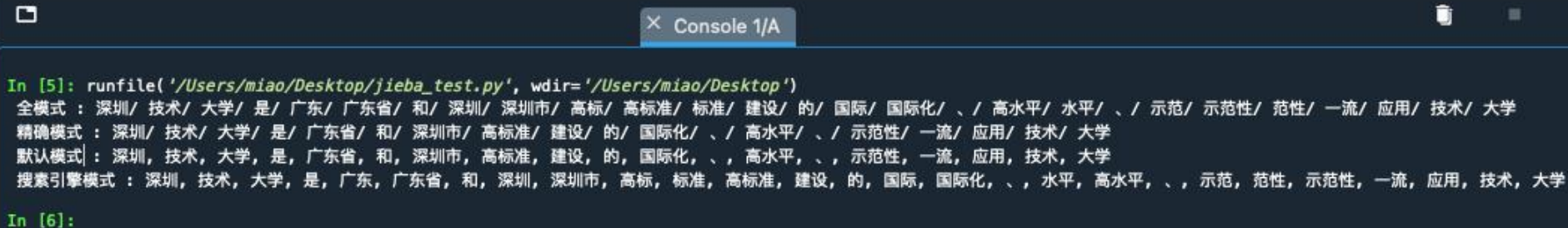
案例分析：

- (1) 导入 jieba 分词工具包
- (2) 使用 `jieba.cut` 方法，在 `cut_all` 参数中分别设置使用全模式和精确模式进行分词；
- 使用 `jieba.cut_for_search` 方法，设置使用搜索引擎模式进行分词。

```
import jieba
# 全模式
seg_list = jieba.cut("深圳技术大学是广东省和深圳市高标准建设的国际化、高水平、示范性一流应用技术大学", cut_all=True)
print(" 全模式 :", "/ ".join(seg_list))
# 精确模式
seg_list = jieba.cut("深圳技术大学是广东省和深圳市高标准建设的国际化、高水平、示范性一流应用技术大学", cut_all=False)
print(" 精确模式 :", "/ ".join(seg_list))
# 默认是精确模式
seg_list = jieba.cut("深圳技术大学是广东省和深圳市高标准建设的国际化、高水平、示范性一流应用技术大学")
print(" 默认模式 :", "/ ".join(seg_list))
# 搜索引擎模式
seg_list = jieba.cut_for_search(" 深圳技术大学是广东省和深圳市高标准建设的国际化、高水平、示范性一流应用技术大学 ")
print(" 搜索引擎模式 :", "/ ".join(seg_list))
```

3.Jieba 分词案例——三种分词结果对比

【运行结果】



```
In [5]: runfile('/Users/miao/Desktop/jieba_test.py', wdir='/Users/miao/Desktop')
全模式：深圳/ 技术/ 大学/ 是/ 广东/ 广东省/ 和/ 深圳/ 深圳市/ 高标/ 高标准/ 标准/ 建设/ 的/ 国际/ 国际化/ 、/ 高水平/ 水平/ 、/ 示范/ 示范性/ 范性/ 一流/ 应用/ 技术/ 大学
精确模式：深圳/ 技术/ 大学/ 是/ 广东省/ 和/ 深圳市/ 高标准/ 建设/ 的/ 国际化/ 、/ 高水平/ 、/ 示范性/ 一流/ 应用/ 技术/ 大学
默认模式：深圳，技术，大学，是，广东省，和，深圳市，高标准，建设，的，国际化，、，高水平，、，示范性，一流，应用，技术，大学
搜索引擎模式：深圳，技术，大学，是，广东，广东省，和，深圳，深圳市，高标，标准，高标准，建设，的，国际，国际化，、，水平，高水平，、，示范，范性，示范性，一流，应用，技术，大学

In [6]:
```

谢谢观看

